

Catálogo de piezas de carro — Fase 1 (catálogo, sin carrito) · Modo oscuro por defecto

Refacciones CALROD

Este documento es el brief técnico completo para que **Claude Code** construya, desde cero, el catálogo web de piezas automotrices de **Refacciones CalRod**: frontend en **Vue 3** con modo oscuro por defecto, datos y fotos en **Supabase** (Postgres + Storage), y despliegue en **Vercel**. La fase actual es solo catálogo (sin carrito ni pagos), pero el esquema de base de datos ya queda listo para que en Fase 2 se agreguen carrito, checkout y cuentas de usuario sin rediseñar las tablas existentes.

Instrucción de uso: pega este PDF (o su contenido) como contexto inicial a Claude Code y pídele que siga el plan de la sección 8 en orden, tarea por tarea.

1. Objetivo del proyecto

Construir un sitio de catálogo de piezas de auto, responsive y de nivel profesional, dirigido a público general (no solo mecánicos): debe permitir buscar y filtrar piezas por categoría, marca y modelo de vehículo compatible, y ver una ficha de detalle clara por pieza. No incluye pagos ni carrito en esta fase.

Alcance de la Fase 1

- Página de catálogo (grid de piezas) con buscador y filtros.
- Página de detalle de pieza (ficha técnica).
- **Base de datos y fotos en Supabase**: tablas de piezas, marca, nombre, código, compatibilidad con vehículos y disponibilidad; fotos de piezas en Supabase Storage (ya no es un JSON mock local).
- Filtro por nombre o código de pieza, resuelto con consultas a Supabase (no solo en el navegador).
- Diseño responsive: móvil, tablet y escritorio.
- **Despliegue en Vercel**, conectado al repositorio del proyecto.
- Sin pagos, sin carrito, sin autenticación de usuarios todavía.

Fuera de alcance (Fase 2 — futuro)

- Carrito de compra y checkout.
- Autenticación de usuarios y cuentas (Supabase Auth queda lista para activarse).
- Pasarela de pago.
- Panel de administración para editar piezas desde una interfaz (por ahora se cargan por seed en Supabase).

2. Stack técnico

Capa	Elección	Motivo
Framework front	Vue 3 (Composition API + <script setup>)	Pedido explícito por el cliente
Build tool	Vite	Estándar actual para Vue 3, arranque rápido; build compatible con Ve
Lenguaje	TypeScript	Tipar el modelo de Pieza evita bugs al crecer el catálogo
Routing	Vue Router	Página de catálogo + página de detalle por slug/ID
Estado global	Pinia	Aunque hoy no hay carrito, deja Pinia listo para Fase 2
Estilos	CSS con variables (design tokens) — ver sección 5	Control total del diseño, sin plantilla genérica
Base de datos	Supabase (Postgres administrado)	Base real sin operar servidor propio; API auto-generada

Fotos de piezas	Supabase Storage (bucket público 'part-images')	Mismo proveedor que la BD, URLs públicas directas
Acceso a datos	supabase-js desde el frontend (ver sección 4.2)	Fase 1 es solo lectura pública: no hace falta backend propio
Despliegue	Vercel	Pedido explícito por el cliente; build estático de Vite se sirve directo
Iconos	SVG propios (ver logo CalRod)	Consistentes con la identidad de marca

3. Arquitectura y estructura de carpetas

Sin backend propio: el frontend habla directo con Supabase (API REST auto-generada sobre Postgres, con permisos de solo lectura pública en Fase 1). Esto simplifica el despliegue a un solo proyecto en Vercel. Estructura pensada para que en Fase 2 solo se agreguen tablas nuevas (cart, orders) y se active Supabase Auth, sin reescribir lo ya construido.

```
calrod/
supabase/
migrations/
0001_init.sql (tablas parts, part_specs, part_compatibility)
0002_rls_policies.sql (permisos de solo lectura publica)
seed.sql (80-150 piezas de ejemplo + specs + compatibilidad)
storage/
part-images/ (bucket publico, subida de fotos de ejemplo aqui)

src/
assets/fonts/ (Sora, Inter, JetBrains Mono)
components/
layout/ AppHeader.vue AppFooter.vue
brand/ LogoMark.vue WrenchWatermark.vue
catalog/
SearchBar.vue FilterBar.vue PartCard.vue PartGrid.vue HeroPanel.vue
part-detail/
PartSpecSheet.vue CompatibilityList.vue
composables/
useParts.ts (consulta Supabase, ya no lee JSON local)
useFilters.ts (arma los filtros: name, code, brand, category)
services/
supabase.ts (createClient con URL + anon key desde variables de entorno)
router/index.ts
stores/catalogStore.ts (Pinia: piezas, filtros activos, estado de carga)
types/part.ts (interface Part, incluye image_url del bucket)
views/
CatalogView.vue PartDetailView.vue NotFoundView.vue
App.vue
main.ts
styles/
tokens.css (variables de diseño – tema oscuro por defecto)
base.css

.env.local (SUPABASE_URL, SUPABASE_ANON_KEY – no se sube al repo)
vercel.json (opcional: config de rewrites/headers si hace falta)
```

4. Base de datos y acceso a datos (Supabase)

4.1 Esquema de base de datos (Postgres, vía migración de Supabase)

Compatibilidad se modela como tabla aparte (una pieza puede servir para varios autos), no como texto libre — así el filtro por modelo/marca es una consulta real. `image_url` guarda la ruta pública del bucket de Supabase Storage.

```
-- supabase/migrations/0001_init.sql

create table parts (
  id uuid primary key default gen_random_uuid(),
  code text unique not null, -- numero de parte / SKU, ej. "BR-4471-C"
  name text not null,
  category text not null, -- frenos | motor | suspension | electrico | carroceria |
  filtros
  brand text not null, -- marca de la refaccion (ej. "CalRod Original")
  origin_type text not null, -- 'original' | 'alternativa' | 'remanufacturada'
  price numeric not null,
  availability text not null, -- 'disponible' | 'stock_bajo' | 'agotado'
  description text,
  material text,
  image_url text, -- URL publica del bucket part-images
  created_at timestamptz default now()
);

create table part_specs (
  id bigint generated always as identity primary key,
  part_id uuid not null references parts(id) on delete cascade,
  label text not null, -- ej. "Diametro"
  value text not null -- ej. "280mm"
);

create table part_compatibility (
  id bigint generated always as identity primary key,
  part_id uuid not null references parts(id) on delete cascade,
  vehicle_brand text not null, -- ej. "Nissan"
  vehicle_model text not null, -- ej. "Sentra"
  year_from int not null,
  year_to int not null
);

create index idx_parts_code on parts(code);
create index idx_parts_name on parts(name);
create index idx_compat_vehicle on part_compatibility(vehicle_brand, vehicle_model);
```

4.2 Permisos (Row Level Security) y acceso desde el frontend

Supabase expone una API REST automática sobre estas tablas. En Fase 1 el catálogo es público de solo lectura: se activa RLS y se agrega una policy de SELECT abierta; no hay policies de INSERT/UPDATE/DELETE públicas (los datos se cargan por `seed.sql` desde el panel de Supabase).

```
-- supabase/migrations/0002_rls_policies.sql

alter table parts enable row level security;
alter table part_specs enable row level security;
alter table part_compatibility enable row level security;

create policy "Lectura publica de piezas" on parts for select using (true);
create policy "Lectura publica de specs" on part_specs for select using (true);
create policy "Lectura publica de compatibilidad" on part_compatibility for select
using (true);
```

Ejemplo de consulta desde el frontend (composable useParts.ts):

```
const { data, error } = await supabase
  .from('parts')
  .select('*', part_specs(*), part_compatibility(*))
  .or(`name.ilike.%${search}%,code.ilike.%${search}%`)
  .eq(category ? 'category' : 'id', category ?? undefined)
```

El filtro por nombre o código (search) usa ilike sobre parts.name y parts.code directamente en la consulta a Supabase — funciona igual sin importar cuántas piezas tenga el catálogo.

4.3 Fotos de piezas (Supabase Storage)

- Crear un bucket público llamado **part-images** en Supabase Storage.
- Subir ahí las fotos de ejemplo (o placeholders) durante el seed; guardar la URL pública resultante en parts.image_url.
- El frontend consume image_url directo con — no necesita lógica extra de carga de archivos en esta fase.

4.4 Datos de ejemplo (seed)

seed.sql llena la base con 80-150 piezas realistas en las 6 categorías, cada una con 1-3 registros de compatibilidad (marca/modelo/años) usando marcas y modelos reales comunes en el mercado: Nissan, Toyota, Chevrolet, VW, Kia, Mazda, Honda. Se corre una sola vez desde el SQL Editor de Supabase o vía Supabase CLI.

5. Sistema de diseño (tokens)

Dirección de diseño: **identidad CalRod llevada a un panel digital premium, modo oscuro por defecto**. Se parte del logo real de la marca (engrane + llave, naranja sobre carbón cálido) y se traduce a una interfaz tipo SaaS profesional — no un e-commerce genérico de fondo blanco, ni tampoco una estética 'rústica de taller'. El naranja de marca es el único acento fuerte; el resto es carbón cálido y crema, igual que la paleta original del logo.

Color

Token	Hex	Uso
--bg	#15130F	Fondo principal (carbón cálido, no negro puro)
--surface	#1C1913	Tarjetas y paneles
--surface-2	#242019	Superficies elevadas (media de tarjeta, hover)
--orange	#E2762A	Acento de marca: CTA, precios, hover, badges 'Original'
--orange-2	#F08C3F	Variante clara del naranja para texto sobre oscuro

--cream	#F2EEDF	Texto principal — mismo crema del fondo del logo original
--charcoal	#8C877C	Trazo del ícono del engrane, texto terciario

El modo oscuro es el único tema de la Fase 1 (no se construye un toggle claro/oscuro todavía).

Tipografía

- **Display** — 'Sora' (600/700/800): logo, títulos, hero, nombres de pieza. Geométrica y con peso, eco del rótulo grueso de 'CalRod' en el logo original.
- **Body** — 'Inter': texto de lectura, descripciones, navegación.
- **Mono/utility** — 'JetBrains Mono': SKUs, precios, specs técnicas — refuerza la lectura tipo panel de datos.

Logo y activos de marca

- Recrear el ícono de engrane + llave del logo original en trazo limpio de línea (SVG), sin la textura 'grunge'/desgastada del logo físico — esa textura no traduce bien a interfaz digital.
- Wordmark: 'Refacciones' en gris/crema tenue arriba, 'CalRod' abajo en dos colores — 'Cal' en crema, 'Rod' en naranja de marca — replicando la jerarquía de color del logo original.
- El ícono del engrane/llave se reutiliza como marca de agua sutil (5% de opacidad) detrás del titular del hero.

Layout y componente firma

El elemento distintivo del sitio es el **panel 'CalRod al día'** en el hero: una tarjeta con tres renglones de estado (piezas activas, marcas cubiertas, % de disponibilidad inmediata), cada uno con un ícono en un chip naranja — comunica confianza y catálogo actualizado, sin recurrir a un dashboard genérico de e-commerce. Las tarjetas de producto llevan badge 'ORIGINAL' o 'ALTERNATIVA' en la esquina, un punto de estado de stock, y un renglón de precio/SKU en monoespaciada separado por línea — legible como una ficha, no como una simple card con sombra difusa.

*Se entrega junto a este PDF el archivo **catalogo_calrod_v3.html** con el mockup visual completo (header con logo real, hero, panel de estado, filtros y grid de tarjetas) para usar como referencia exacta de implementación — colores, tipografía y spacing deben copiarse de ahí.*

6. Páginas y comportamiento funcional

CatalogView (/)

- Hero con titular + HeroPanel.vue ('CalRod al día': piezas activas, marcas cubiertas, % disponibilidad — estos números vienen de consultas a Supabase, no están escritos a mano).
- WrenchWatermark.vue como fondo sutil del hero, detrás del titular (5% opacidad, se oculta en móvil).
- Buscador (SearchBar.vue): al escribir, consulta Supabase (ilike sobre name/code) con debounce (~300ms).
- FilterBar.vue: chips de categoría (multi-selección opcional) + contador de resultados.
- PartGrid.vue: grid responsive (auto-fill, mínimo ~255px por tarjeta).
- Estado vacío: si el filtro no encuentra nada, mostrar mensaje claro invitando a limpiar filtros.

PartDetailView (/pieza/:id)

- Ficha completa: imagen, SKU, nombre, precio, descripción, tabla de especificaciones (specs[]).
- Lista de compatibilidad (marca + modelos) en formato claro, tipo ficha técnica.
- Botón 'Volver al catálogo' — sin botón de compra en esta fase (dejar el slot listo para Fase 2).

Responsive

- Escritorio: hero a 2 columnas, grid de 3-4 tarjetas por fila.
- Tablet: hero apila, grid de 2 tarjetas por fila.
- Móvil: navegación colapsa, buscador ocupa todo el ancho, grid de 1 columna, filtros con scroll horizontal.

7. Requisitos no funcionales

- Accesibilidad: foco visible en elementos interactivos, contraste AA mínimo, atributos alt en imágenes.
- Rendimiento: debounce en la búsqueda; imágenes con lazy-load (ya sirven desde el CDN de Supabase Storage).
- SEO básico: título y meta description por vista, usando vue-router meta fields.
- La anon key de Supabase es pública por diseño (queda protegida por las políticas de RLS de solo lectura); nunca se expone una service role key en el frontend.
- Manejo de errores: si Supabase no responde, el frontend muestra un estado de error claro, no una pantalla en blanco.
- Código preparado para Fase 2: la tienda entra sin romper componentes existentes (cart store aparte, botón de compra como slot/placeholder ya reservado en PartCard y PartDetailView; tablas de carrito/usuarios se agregan sin tocar parts/part_specs/part_compatibility, y Supabase Auth se activa cuando haga falta).

8. Despliegue en Vercel

- Conectar el repositorio del proyecto a un nuevo proyecto de Vercel (framework preset: Vite).
- Variables de entorno en Vercel (Project Settings → Environment Variables): VITE_SUPABASE_URL y VITE_SUPABASE_ANON_KEY, tomadas del panel de Supabase (Project Settings → API).
- Build command: `npm run build` — Output directory: `dist` (configuración estándar de Vite, Vercel la detecta sola).
- Cada push a la rama principal genera un deploy de producción; cada pull request genera un preview URL aparte — útil para revisar cambios de diseño antes de publicarlos.
- El dominio del catálogo (o un subdominio `*.vercel.app` mientras tanto) se configura desde Project Settings → Domains una vez que el cliente tenga uno propio.

9. Plan de ejecución para Claude Code

Ejecutar en este orden. Cada tarea debe terminar en un estado que compila y corre (`npm run dev`) antes de pasar a la siguiente.

TAREA 1 — Proyecto de Supabase

- Crear el proyecto en supabase.com (region cercana al mercado del cliente).
- Correr `supabase/migrations/0001_init.sql` (tablas) y `0002_rls_policies.sql` (policies de lectura publica).
- Crear el bucket publico 'part-images' en Supabase Storage.
- Guardar `SUPABASE_URL` y `SUPABASE_ANON_KEY` para usarlos como variables de entorno.

TAREA 2 — Datos de ejemplo (seed)

- Subir 80-150 fotos de ejemplo (o placeholders) al bucket part-images.
- Correr `seed.sql` desde el SQL Editor de Supabase: 80-150 piezas realistas en las 6 categorias, con specs y 1-3 registros de compatibilidad cada una (marcas/modelos reales: Nissan, Toyota, Chevrolet, VW, Kia, Mazda, Honda).
- Verificar en el Table Editor de Supabase que los datos y las URLs de imagen quedaron bien.

TAREA 3 — Andamiaje del frontend

- Crear proyecto con Vite (plantilla vue-ts). Instalar Vue Router, Pinia y `@supabase/supabase-js`.
- Crear la estructura de carpetas de la seccion 3 y el archivo `.env.local` con las variables de Supabase.
- Cargar las 3 fuentes (Sora, Inter, JetBrains Mono) via `@font-face` o Google Fonts.
- Crear `src/styles/tokens.css` con las variables de color de la seccion 5 (tema oscuro por defecto).
- Crear `services/supabase.ts` con `createClient()` usando las variables de entorno.

TAREA 4 — Marca y layout base

- Construir `LogoMark.vue` (SVG engrane + llave, trazo limpio) replicando `catalogo_calrod_v3.html`.
- Construir `AppHeader.vue` (LogoMark + wordmark 'Refacciones CalRod', nav, `SearchBar`) y `AppFooter.vue`.
- Montar `App.vue` con `router-view`.

TAREA 5 — Hero y panel de estado

- Construir `WrenchWatermark.vue` y `HeroPanel.vue` — los 3 numeros del panel vienen de consultas a Supabase (`count de parts`, `count distinct de brand`, `% availability = disponible`), no `hardcodeados`.
- Armar el hero completo (titular + CTA + `HeroPanel`) segun `catalogo_calrod_v3.html`.

TAREA 6 — Catálogo (grid + filtros + buscador conectados a Supabase)

- Construir `useParts.ts` y `useFilters.ts`: arman la consulta a Supabase (seccion 4.2) segun los filtros activos.
- Construir `FilterBar.vue` (chips de categoria, con conteo por categoria) y `SearchBar.vue` con `debounce`.
- Construir `PartCard.vue` (badge de `origin_type`, punto de disponibilidad, SKU en mono, imagen desde `image_url`) y `PartGrid.vue`.
- Construir `CatalogView.vue` integrando todo, con estado de carga y estado vacio/error de red.

TAREA 7 — Página de detalle

- Construir PartSpecSheet.vue y CompatibilityList.vue consumiendo la consulta con part_specs y part_compatibility embebidos.
- Construir PartDetailView.vue con ruta /pieza/:id y manejo de 404 (NotFoundView.vue) cuando Supabase no encuentra la pieza.

TAREA 8 — Responsive y pulido

- Revisar los 3 breakpoints (movil/tablet/escritorio) descritos en la seccion 6.
- Verificar foco de teclado visible y contraste de color.
- Revisar que no haya CSS con selectores que se pisen entre si (misma clase con !important repetido).

TAREA 9 — Despliegue en Vercel

- Conectar el repositorio a Vercel y configurar VITE_SUPABASE_URL / VITE_SUPABASE_ANON_KEY como variables de entorno (seccion 8).
- Verificar el deploy de produccion y confirmar que el catalogo carga datos reales desde Supabase.

TAREA 10 — Entrega

- README.md explicando como correr el proyecto en local, donde estan las migraciones/seed de Supabase, y como se despliega en Vercel.
- Dejar comentado en el codigo donde entraria el carrito/checkout de Fase 2 (sin implementarlo).

10. Notas para Fase 2 (no construir ahora)

- cartStore.ts en Pinia + CartDrawer.vue.
- Vista de checkout y confirmacion de pedido.
- Tablas orders, order_items y activacion de Supabase Auth para cuentas de usuario.
- Policies de RLS adicionales para que cada usuario solo vea/edite sus propios pedidos.
- Panel de administracion (o Supabase Studio con roles) para editar piezas sin tocar SQL directo.